Desafio Software Engineer

_cultura itubers

Valores que guiam a jornada de transformação cultural

a gente_ trabalha para o cliente

a gente_ é movido por resultado

pra gente_ ética é inegociável

a gente_ não sabe tudo

a gente_ quer diversidade

a gente_ vai de turma

_Bem Vindo

Obrigado por participar do nosso processo! 😉

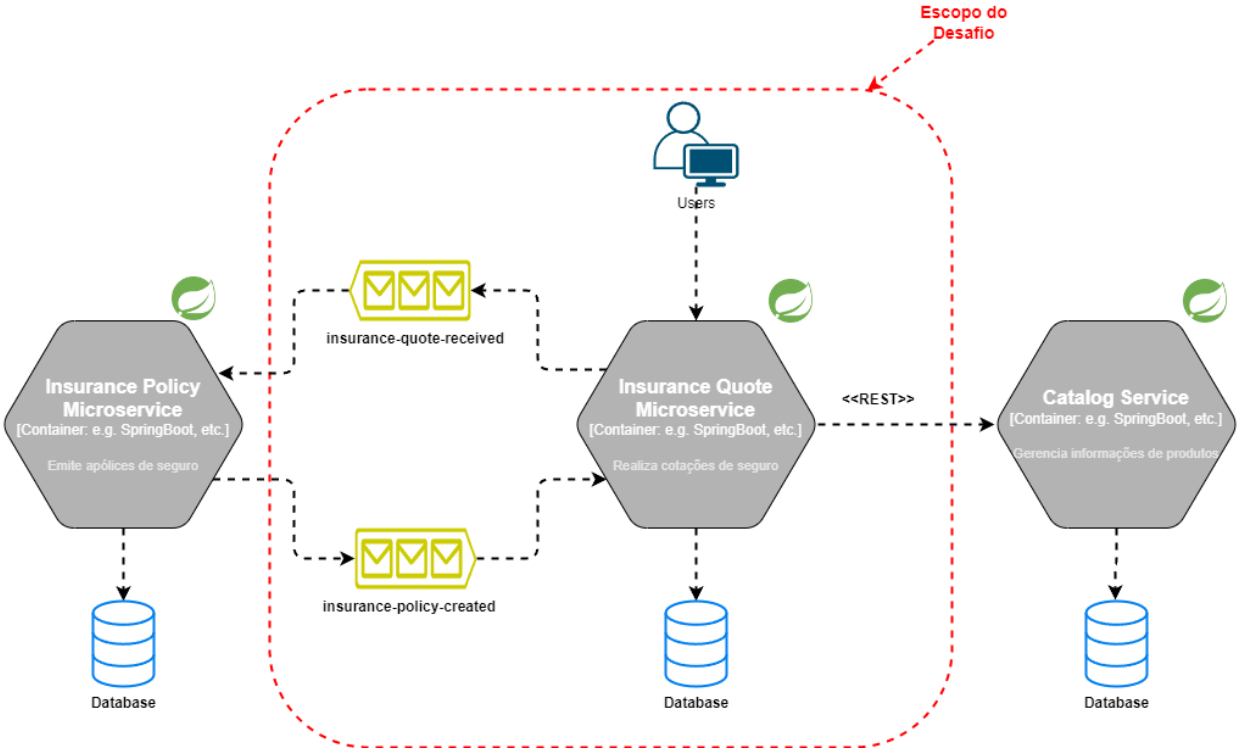
_Preparando o Ambiente

Tecnologias necessárias para o desafio:

- JDK11 ou JDK17
- Docker ou Kubernetes
- Maven ou Gradle
- IntelliJ ou Eclipse (ou qualquer outra IDE de sua preferência)
- Postman ou Insomnia (ou qualquer outra ferramenta de sua preferência)

_Resumo

A seguradora ACME precisa de uma API REST capaz de receber e consultar cotações de seguro interagindo com outros sistemas conforme o desenho abaixo:



Representação do Desafio

O corpo da requisição para solicitação de cotação de seguro deverá conter os seguintes campos:

- ID do produto
- ID da oferta

- Categoria
- Total do prêmio mensal
- Total do valor de coberturas (somatória)
- Lista de coberturas contendo o nome e valor de cada uma
- Lista de assistências do seguro
- Dados do Cliente

Abaixo um exemplo da requisição contendo os campos descritos:

```
{
    "product_id": "1b2da7cc-b367-4196-8a78-9cfeec21f587",
    "offer_id": "adc56d77-348c-4bf0-908f-22d402ee715c",
    "category": "HOME",
    "total_monthly_premium_amount": 75.25,
    "total_coverage_amount": 825000.00,
    "coverages": {
      "Incêndio": 250000.00,
      "Desastres naturais": 500000.00,
      "Responsabiliadade civil": 75000.00
    },
    "assistances": [
      "Encanador",
      "Eletricista",
      "Chaveiro 24h"
    ],
    "customer": {
        "document_number": "36205578900",
        "name": "John Wick",
        "type": "NATURAL",
        "gender": "MALE",
        "date_of_birth": "1973-05-02",
        "email": "johnwick@gmail.com",
        "phone_number": 11950503030
    }
}
```

Ao receber a requisição é necessário validar a oferta e o produto informados consultando a API do serviço de Catálogo.

Você pode criar um **mock server** para simular o retorno desta API com os dados que forem necessários.

Exemplo do retorno da consulta de produtos:

```
{
    "id": "1b2da7cc-b367-4196-8a78-9cfeec21f587",
    "name": "Seguro de Vida",
    "created_at": "2021-07-01T00:00:00Z",
    "active": true,
```

```
    "offers": [
        "adc56d77-348c-4bf0-908f-22d402ee715c",
        "bdc56d77-348c-4bf0-908f-22d402ee715c",
        "cdc56d77-348c-4bf0-908f-22d402ee715c"
    ]
}
```

Exemplo do retorno da consulta de ofertas:

```
{
    "id": "adc56d77-348c-4bf0-908f-22d402ee715c",
    "product_id": "1b2da7cc-b367-4196-8a78-9cfeec21f587",
    "name": "Seguro de Vida Familiar",
    "created_at": "2021-07-01T00:00:00Z",
    "active": true,
    "coverages": {
        "Incêndio": 500000.00,
        "Desastres naturais": 600000.00,
        "Responsabiliadade civil": 80000.00,
        "Roubo": 100000.00
      },
      "assistances": [
        "Encanador",
        "Eletricista",
        "Chaveiro 24h",
        "Assistência Funerária"
      ],
      "monthly_premium_amount": {
        "max_amount": 100.74,
        "min_amount": 50.00,
        "suggested_amount": 60.25
      }
}
```

A seguradora ACME preza muito pela **experiência do cliente**, portanto seria interessante se houvesse algum tipo de **cache** durante a **consulta** dos produtos e ofertas no momento da validação.

Os seguintes itens devem ser validados:

- O produto e oferta são **existentes** e estão **ativos**
- As **coberturas informadas** estão dentro da lista de coberturas da oferta e abaixo dos valores **máximos permitidos**.
- As **assistências informadas** estão dentro da lista de assistências da oferta.
- O **valor total do prêmio mensal** está entre o **máximo** e **mínimo** definido para a oferta.

- O **valor total das coberturas** corresponde a somatória das coberturas informadas corresponde ao **valor total informado das coberturas** no corpo da requisição.
- Os campos dos dados do cliente são **livres**.

Caso a solicitação seja **válida** é necessário persistir a cotação em um **banco de dados** de sua preferência gerando um identificador único em formato **numérico** e enviar uma mensagem de cotação recebida no **broker** de sua **preferência**.

Se a solicitação for **inválida** é necessário retornar um **erro** na chamada da API para que o cliente corrija os dados e tente novamente.

Após processar a mensagem da **cotação recebida** o serviço de apólices irá emitir a apólice desta cotação e enviar uma mensagem com as informações da **apólice emitida**.

São necessários minimamente os campos do **ID da apólice**, **ID da cotação** e **data e hora de emissão**, sinta-se livre para adicionar outros campos.

O serviço de cotação deverá então receber esta mensagem e **atualizar a cotação** com o número da apólice emitida, portanto é necessário também que exista um endpoint para **consulta de cotações** que retornem dados conforme o exemplo abaixo:

```
{
    "id": 22345,
    "insurance_policy_id": 756969,
    "product_id": "1b2da7cc-b367-4196-8a78-9cfeec21f587",
    "offer_id": "adc56d77-348c-4bf0-908f-22d402ee715c",
    "category": "HOME",
    "created_at": "2024-05-22T20:37:17.090098",
    "updated_at": "2024-05-22T21:05:02.090098",
    "total_monthly_premium_amount": 75.25,
    "total_coverage_amount": 825000.00,
    "coverages": {
      "Incêndio": 250000.00,
      "Desastres naturais": 500000.00,
      "Responsabiliadade civil": 75000.00
    },
    "assistances": [
      "Encanador",
      "Eletricista",
      "Chaveiro 24h"
    ],
    "customer": {
        "document_number": "36205578900",
        "name": "John Wick",
        "type": "NATURAL",
        "gender": "MALE",
        "date_of_birth": "1973-05-02",
```

```
        "email": "johnwick@gmail.com",
        "phone_number": 11950503030
    }
```

Na seção abaixo indicaremos alguns brokers que podem ser utilizados para que você possa simular tanto simular **manualmente** o envio da mensagem pelo serviço de apólices como também confirmar a geração da mensagem do serviço de cotações.

Caso deseje você pode fazer este processo de forma automatizada com alguma **aplicação/framework** de sua preferência (o objetivo é que você possa testar de forma integrada).

_Orientações Gerais

- Não é necessário desenvolver o serviço de apólices nem o de catálogo (para o serviço de **catálogo** sugerimos um **mock server** como descrito anteriormente e para o serviço de **apólices** sugerimos utilizar algum broker que permita o envio/recebimento de mensagens através de uma **interface gráfica** ou **CLI**).
- Crie um arquivo **docker-compose.yml** com toda a infraestrutura necessária para rodar seu desafio tais como banco de dados, broker de mensageria, mock server e sua aplicação.
- Não esqueça de documentar no **README.md** o método escolhido para interação com as mensagens com exemplos de utilização. 😉

Deixamos como alternativas os seguintes brokers de mensageria:

- Apache Kafka
- RabbitMQ
- ActiveMQ
- AWS SQS

Você pode escolher o broker que se sentir mais confortável para o desafio utilizando mensagens em formato **XML**, **JSON**, **Avro**, **Protobuf** ou outros que preferir.

_Pontos que daremos mais atenção

Abaixo os pontos que daremos mais atenção na entrega do desafio:

- Testes de unidade e integração
- Cobertura de testes (Code Coverage)
- Arquitetura utilizada
- Abstração, acoplamento, extensibilidade e coesão
- Profundidade na utilização de Design Patterns
- Clean Architecture
- Clean Code
- SOLID
- Documentação da Solução no README.md

- Observabilidade (métricas, traces e logs)

_Pontos que não iremos avaliar

Abaixo os pontos que não iremos avaliar, mas caso queira adicionar no desafio seria interessante:

- Dockerfile
- Scripts CI/CD
- Collections do Postman, Insomnia ou qualquer outra ferramenta para execução

_Sobre a documentação

Nesta etapa do processo seletivo queremos entender as decisões por trás do código, portanto é fundamental que o README.md tenha algumas informações referentes a sua solução.

Algumas dicas do que esperamos ver são:

- Instruções básicas de como executar o projeto
- Detalhes sobre a solução, gostaríamos de saber qual foi o seu racional nas decisões
- Caso algo não esteja claro e você precisou assumir alguma premissa, quais foram e o que te motivou a tomar essas decisões

_Como esperamos receber sua solução

Esta etapa é eliminatória, e por isso esperamos que o código reflita essa importância.

Se tiver algum imprevisto, dúvida ou problema, por favor entre em contato com a gente, estamos aqui para ajudar.

Atualmente trabalhamos com a stack Java/Spring, porém você pode utilizar a tecnologia de sua preferência.

Para candidatos externos nos envie o link de um repositório público com a sua solução e para candidatos internos o projeto em formato .zip

_Observações importantes

Não é necessário parametrizar os impostos em arquivos de configuração ou persisti-los em base de dados.

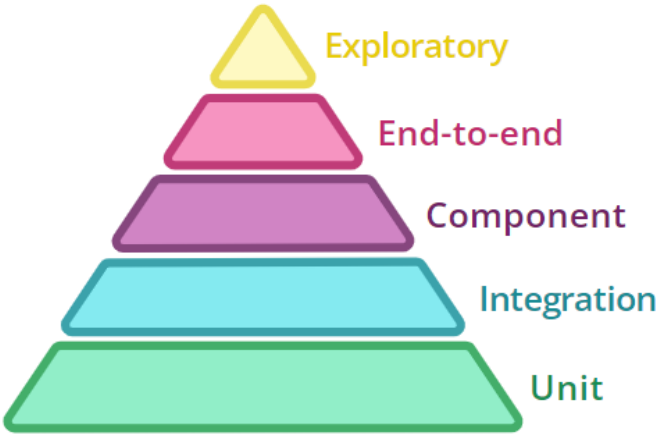
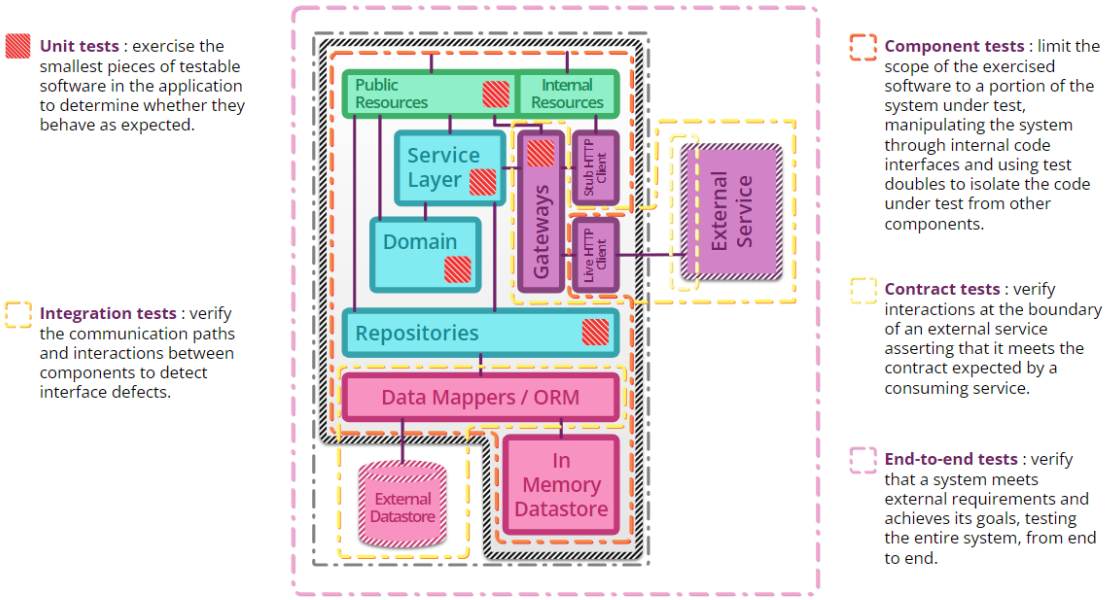
Os campos a serem persistidos devem ser somente os informados no desafio.

_Dicas

Aqui vão algumas dicas que podem ser úteis:

Como item opcional de leitura, deixamos este artigo rápido sobre testes Testing Strategies in a Microservice Architecture

Nele é possível ver a diferença entre os principais tipos de teste.



Também há um exemplo para cada tipo de teste no artigo que pode ajudar no desafio.